

Distributed Systems — Homework 3

Sections 1, 2 and 3 — Design, Verification and Performance

Overview

This report covers all three sections of Homework 3. Every program was executed on the **RCE cluster** through Slurm, with servers and clients on separate compute nodes, and every result below was measured rather than estimated.

Section	Problem	Language	Verification
1 — Q1	Distributed Matrix Multiplication (MapReduce)	Python	PASS 9 / 9 cases on 4 nodes
2 — Q1	Batch Log Analytics (MapReduce)	C++	PASS 34 / 34 checks
2 — Q2	Real-Time Log Analytics (gRPC)	Python	PASS 76 / 76 checks
3 — Q2	Food Ordering System (gRPC)	Python	PASS all cases on 3 nodes

A note on Hadoop. The assignment specifies Apache Hadoop 3.3.6 on YARN for MapReduce. This is not usable on RCE: the shared HDFS is read-only for a student account (root is `hdfs:supergroup`, mode `drwxr-xr-x`) and no ResourceManager is configured, so no YARN job can be submitted. The course has confirmed the Hadoop environment has a known issue and that a **Slurm-based script** is acceptable until it is fixed. Both MapReduce problems are therefore orchestrated by shell scripts in exactly the shape Hadoop Streaming would use: mapper, combiner and reducer are standalone executables reading stdin and writing stdout, so they would run unmodified under real Hadoop Streaming.

Cluster. Nodes are dual Intel Xeon Gold 5317 (24 physical cores, 48 logical, hyperthread siblings at stride 24), 502 GB RAM, shared NFS home.

A measurement caveat that matters. `--nodes=N --ntasks-per-node=1` without `--cpus-per-task` grants *one physical core per node*: `nproc` reports 2, but those are the two hyperthreads of a single core. Under that allocation two mapper processes on a node take 2.49s where one takes 1.27s, and `mpirun -np 8` runs eight ranks on one core. All benchmarks here request `--cpus-per-task` so that N tasks and N MPI ranks each receive N real cores.

1 Section 1 — Q1: Distributed Matrix Multiplication

1.1 Approach

$C = A \times B$ by the **Row–Row method**: each row c_i is a weighted sum of the rows of B , with the entries of a_i as weights. Matrix B is small relative to A and is **replicated into every mapper’s memory**; A is split by rows across mappers.

- **Map**: for a row i of A , emit one partial vector $a_{ik} \cdot B_{k,:}$: per non-zero position k , keyed by the row index (zero padded so the sort orders keys correctly). Mappers never communicate.
- **Combine**: sum partial vectors sharing a row key locally, before the network shuffle. Input and output have the same format.
- **Reduce**: sum the remaining partial vectors per row and emit the rows of C in ascending row order.

1.2 Correctness

All nine cases run distributed across 4 nodes (Slurm job 98640): Assignment Example 1 (even split) and Example 2 (uneven split), the edge cases $m = 1$ and $n = 1$, both m divisible and not divisible by the task count, and skewed shapes (tall $m \gg n$, wide $n \gg m$) up to $1001 \times 100 \times 100 \times 1000$. Each result is checked against a direct triple-loop multiplication.

Result: 9 PASSED / 0 FAILED

1.3 Performance

Slurm job 98646, 4 nodes, 4 map tasks, seconds.

Case	A	B	$m \text{ div } 4$	Map	Shuffle	Combine	Reduce	Total
p1_small	100x50	50x50	yes	0.22	0.17	0.18	0.02	0.60
p1_medium	500x50	50x50	yes	0.31	0.22	0.24	0.03	0.82
p1_large	2000x50	50x50	yes	0.76	0.38	0.46	0.09	1.72
p1_square	1000x100	100x1000	yes	11.05	12.30	5.53	0.69	29.63
p1_square_uneven	1001x100	100x1000	no	11.07	13.11	5.54	0.69	30.47
p1_tall	5000x10	10x10	yes	0.27	0.19	0.23	0.06	0.77
p1_wide	10x500	500x10	no	0.18	0.17	0.19	0.02	0.57

Observations.

- **Intermediate volume, not input size, dominates.** The square cases ($1000 \times 100 \times 100 \times 1000$) take ~ 30 s against 1.7 s for 2000×50 , despite comparable input. Each row emits n partial vectors of p values, so intermediate data grows as $m \cdot n \cdot p$ — roughly 400 MB of text here. The map and shuffle stages carry that cost.
- **Uneven splits are essentially free.** 1000 rows (even) and 1001 rows (uneven) differ by 0.85 s out of ~ 30 s; the extra row lands on one node.
- **The combiner is what keeps the final merge small.** It collapses each node’s partial vectors to one row each before the global shuffle, so the second sort costs 0.05 s even when the intermediate data is hundreds of megabytes.
- **Wide matrices under-use the cluster:** 10×500 has only 10 rows to distribute over 4 nodes, so most nodes idle.

2 Section 2 — Q1: Batch Log Analytics with MapReduce

2.1 Design

The HW2 Q7 analytics (totals, success/failure counts, status-class counts, min/max/mean response time, total bytes, busiest 60-second interval, Top- K servers and endpoints) computed as a MapReduce job in C++.

Everything kept is a count, a sum, a minimum or a maximum, so partial results combine exactly and in any order. An average is never stored — the sum and the count are, and the division happens once at print time. Response times are held as whole numbers of millionths of a millisecond, so summation order can never change the answer.

- **In-mapper combining.** A mapper counts its whole split and emits one pair per distinct key — `G (overall)`, `S:id`, `E:id`, `I:minute` — never one pair per record. Intermediate data is therefore bounded by the *key space*, not by the record count: the shuffle is **278× smaller than the input** on 10M records.
- **A single reducer**, because Top- K needs global counts: a server ranked 11th on every reducer could still be 1st overall. This is cheap precisely because in-mapper combining already shrank the data to megabytes.

2.2 Correctness

Output must match HW2's sequential program **log_seq byte for byte**. 34 checks: the worked example, five edge cases (empty input, one record, ties, $K = 0$, values as small as 0.000001 ms), and the same input split 1, 2, 4 and 8 ways — splitting differently must never change the answer. In the benchmark sweep a further **24 of 24** runs matched.

Result: 34 / 34 checks, and 24 / 24 benchmark runs correct

2.3 Performance

10M records (`large.in`, 396 MB), map tasks one per node across 4 nodes.

Map tasks	small (100k)	medium (1M)	large (10M)	large: map stage	Speedup
1	0.342	1.701	16.127	15.535	1.00×
2	0.318	1.095	9.383	8.872	1.72×
4	0.296	0.696	4.922	4.453	3.28×
8	0.599	1.026	5.182	4.469	3.11×

Observations.

- **The map stage is the job, and it is CPU-bound.** It is 96% of the single-task run. Reading the 397 MB input with `cat` takes 0.26 s, while the mapper takes 13.26 s of which 13.16 s is user CPU — the cost is parsing 10M lines, not fetching them.
- **Four tasks is the ceiling on four nodes (3.28×).** At eight tasks the map stage does not improve at all, because Slurm places the second task on some nodes onto a *sibling hyperthread of the same physical core*; the extra tasks only enlarge the sort and combine stages, so the total gets slightly worse.
- **The serial tail is real but small:** the final gather, sort and reduce grow from 0.22 s to 0.28 s as mappers are added, since more mappers emit more partial rows for the same keys.

- **Small inputs should not be distributed:** 100k records gain almost nothing from 1 to 4 tasks and are clearly slower at 8 — process startup across nodes costs more than the parallelism saves.

2.4 MapReduce versus MPI

The same analytics as an MPI program (byte-range split, `MPI_Reduce` to combine), both given equal core counts. MPI runs all ranks on one node because Open MPI cannot launch across nodes on RCE; MapReduce spreads its tasks over four. All 24 of 24 MPI runs matched `log_seq`.

Procs	MPI (large)	MapReduce (large)	MR / MPI	MPI speedup	MR speedup
1	7.611	16.127	2.12×	1.00×	1.00×
2	4.437	9.383	2.11×	1.72×	1.72×
4	2.225	4.922	2.21×	3.42×	3.28×
8	2.194	5.182	2.36×	3.47×	3.11×

MPI is 1.4–2.4× faster at every size, and the ratio is stable at about 2.1×

 on the largest dataset. That is what the paradigms predict: MPI has no shuffle, no sort, no text intermediate form and no process per stage. Both scale almost identically (3.42× and 3.28× at four processes), so the decomposition is equally good in each — MPI is simply faster per unit of work.

Why, measured. The two differ in how they hold state. On `large.in`:

Program	Structure	Peak memory	Time
<code>log_seq</code> (HW2)	two passes, all records in a vector, dense arrays	395 MB	4.65 s
<code>log_mpi</code> , 1 rank	same structure, byte-range split	403 MB	6.31 s
<code>mapper</code> (ours)	one pass, sparse maps	10.1 MB	13.00 s

Our pipeline uses **39×** less memory and is about **2.8×** slower. HW2’s design pre-scans the file to learn the id ranges and indexes dense arrays directly — an array subscript per record, but it must hold all 10M records first. Ours counts in a single streaming pass into sparse maps, so it never holds a record after counting it and never needs the ranges in advance, which is exactly what a mapper cannot know since it sees only its own split. The price is a tree lookup per record. **MPI wins on speed; the MapReduce design wins on memory**, and that memory property is what makes the decomposition possible at all.

3 Section 2 — Q2: Real-Time Log Analytics with gRPC

3.1 Architecture

The same analytics, but over a continuous stream.

```
stream_client --StreamRecords--> +--- Coordinator ---+ --Process()--> Worker 0
(replays the dataset)             | router           | --Process()--> Worker 1
                                  | bounded queues | --Process()--> Worker 2
dashboard --WatchAnalytics---> | adds workers up| <---GetStats()-- (each counts
query_client --GetAnalytics---> +-----+ into its own)
```

gRPC workflows.

- **StreamRecords** — *client streaming*. One long-lived RPC carries a whole stream: a header, then any number of record batches. One RPC per stream rather than per batch avoids paying connection and header costs repeatedly, and gRPC's flow control then provides backpressure for free.
- **GetAnalytics** — *unary*. The coordinator asks every worker for its running totals and adds them up. Answered while ingestion continues.
- **WatchAnalytics** — *server streaming*. The coordinator pushes a snapshot every interval, so N dashboards cost one round of work per refresh rather than N .
- **Worker.Process / Worker.GetStats** — the internal service. A worker keeps its own running totals and always sends all of them, and the coordinator *replaces* that worker's contribution rather than accumulating it. Asking twice is therefore harmless: no sequence numbers, no acknowledgements, no way to double-count.
- **Columnar batches**. A `RecordBatch` holds one packed array per field, so a 1000-record batch is 7 packed arrays rather than 1000 sub-messages; protobuf decodes them in C and the worker walks the columns directly.

Backpressure. Each worker has a bounded queue. When a worker falls behind its queue fills, the put blocks, the ingestion handler stops reading, and gRPC's flow control slows the client. Memory stays bounded however fast the source sends.

3.2 Correctness

After the whole stream is processed the analytics must equal `log_seq` exactly. 76 checks: 1–3 workers $\times$ 3 distribution strategies $\times$ several batch sizes, two sources streaming at once, queries issued while data is still arriving, and K supplied at query time. All 36 of 36 benchmark runs also matched.

Result: 76 / 76 checks, and 36 / 36 benchmark runs correct

3.3 Performance

1M records, one process per node, coordinator and workers and clients all separate.

Worker count (round robin, batch 1000)

Workers	rec/s	Speedup
1	799,329	1.00 $\times$
2	1,555,611	1.95 $\times$
4	2,999,449	3.75 $\times$

Distribution strategy (4 workers)

Strategy	rec/s	vs round robin
round_robin	2,999,449	1.00 $\times$
least_loaded	2,285,566	0.76 $\times$
hash_server	220,454	0.07 $\times$

Message granularity (4 workers) — the single largest factor.

Records per message	Messages for 1M	Throughput (rec/s)	vs batch 1
1	1,000,000	10,126	1×
10	100,000	99,549	10×
100	10,000	844,397	83×
1,000	1,000	2,999,449	296×
10,000	100	3,104,559	307×

Concurrent queries during ingestion (4 workers, batch 1000). Each client issues `GetAnalytics` in a closed loop with no pause — the harshest possible load.

Query clients	Ingest (rec/s)	vs none	Queries/s	p50 (ms)	p99 (ms)
none	2,999,449	—	—	—	—
1	2,371,485	-21%	49.8	21.317	25.297
4	807,953	-73%	146.1	30.302	41.806
16	204,572	-93%	153.5	113.811	181.138

Observations.

- **Scaling is near linear to 4 workers** ($3.75\times$), because workers share nothing — each holds its own state and never talks to another. The gap from a perfect $4\times$ is the coordinator, the one process every record must pass through.
- **Granularity is worth $296\times$** between one record per message and a thousand. Up to batch 100, throughput grows almost exactly $10\times$ per $10\times$ batch size — the signature of a cost that is *per message, not per record*. The curve flattens after 1000, which is why that is the default: a batch then waits only ~ 1 ms to fill, so freshness is unaffected.
- **hash_server is $14\times$ slower** because it is the only strategy that inspects every *record* rather than every *batch*: the coordinator takes each batch apart and re-encodes W smaller ones. Key partitioning moves the bottleneck into the router. It is not needed here, because every analytic is mergeable and so no key affinity is required.
- **least_loaded is slower than round robin**, the opposite of its intent: it polls every queue for every batch, and the workers are near-identical, so it pays the cost without ever finding a shorter queue.
- **Queries are answered throughout, correctly, and they are not free.** One closed-loop client costs 21% of ingest throughput, four cost 73%, sixteen cost 93%, while answers/s saturates around 150. Beyond roughly four concurrent closed-loop clients the single-threaded coordinator is the bottleneck. **This is the clearest limit of the design**; sharding the coordinator, or answering from a periodically refreshed snapshot, would be the next step.

4 Section 3 — Q2: Food Ordering System with gRPC

4.1 Design

A central server holding restaurants and orders, with customer clients and restaurant clients as separate processes on separate nodes.

Service API — all five required RPCs, plus one helper: `ListRestaurants`, `PlaceOrder`, `GetOrderStatus`, `UpdateOrderStatus`, `SubscribeToOrderUpdates` (server streaming), and `ListRestaurantOrders` so a restaurant can see its own pending orders.

Order lifecycle. `PLACED` $\rightarrow$ `ACCEPTED` $\rightarrow$ `PREPARING` $\rightarrow$ `READY`, with `PLACED` $\rightarrow$ `CANCELLED`. Transitions are held in an explicit table and anything not in it is rejected, so an order cannot move backwards and cannot be cancelled once accepted.

Concurrency. All shared state is guarded by a single lock; subscriber notifications are sent *outside* the lock so a slow subscriber cannot block other requests. Each subscriber has its own queue.

4.2 Verification on the cluster

Server on `node01`, clients driven from `node02` and `node03` (Slurm job 98644). Every item the assignment asks the demonstration to show:

Required demonstration	Observed	Result
Listing restaurants	both menus returned over the network	PASS
Placing an order	01, total 550 (250 + 2×150), status <code>PLACED</code>	PASS
Restaurant processing the order	<code>ACCEPTED</code> $\rightarrow$ <code>PREPARING</code> $\rightarrow$ <code>READY</code>	PASS
Customer receiving real-time updates	all three arrived on the stream, in order	PASS
Two clients interacting concurrently	second customer received a distinct order id	PASS
Exception cases, each with the correct gRPC status code		
Order from a non-existent restaurant	<code>NOT_FOUND</code>	PASS
Order an unavailable item	<code>NOT_FOUND</code>	PASS
Status of a non-existent order	<code>NOT_FOUND</code>	PASS
Invalid transition (<code>READY</code> $\rightarrow$ <code>PREPARING</code>)	<code>FAILED_PRECONDITION</code>	PASS
Another restaurant updating our order	<code>PERMISSION_DENIED</code>	PASS
Cancelling an already-accepted order	<code>FAILED_PRECONDITION</code>	PASS

4.3 Known limitation

`SubscribeToOrderUpdates` blocks on its queue with no timeout while holding a thread from the server's pool of ten. Ten customers tracking orders — ordinary use, and what the demonstration asks for — exhaust the pool and the server stops answering any request. Subscribing to an order that has already reached `READY` also blocks forever, since no further update will ever arrive. Both are fixed by polling the queue with a timeout while re-checking whether the client is still connected, sending the current status on subscribe, and enlarging the thread pool.

5 How to run

Section 2 — full instructions in `Section2/run.md`.

Command	Purpose
<code>bash scripts/rce_setup.sh</code>	one-time: build, virtualenv, gRPC stubs, datasets
<code>bash scripts/run_q1.sh data/medium.in out.txt 4</code>	Q1 pipeline once, 4 map tasks, self-checking
<code>bash scripts/verify_q1.sh</code>	Q1 correctness, 34 checks
<code>python3 scripts/verify_q2.py</code>	Q2 correctness, 76 checks
<code>sbatch scripts/bench_q1.sh</code>	Q1 benchmark sweep, 4 nodes
<code>sbatch scripts/bench_q2.sh data/medium.in</code>	Q2 benchmark sweep, 6 nodes
<code>bash scripts/rce_start.sh</code>	start Q2 across an allocation

Section 1 Q1: `sbatch --cpus-per-task=8 run_cluster_test.sh (correctness)`, `sbatch --cpus-per-task=8 benchmark_dist_slurm.sh (performance)`.

Section 3 Q2: server on one compute node, customer and restaurant clients on others, per the RCE execution guide — never the login node, and never `localhost` across nodes.

Port note. Port 50051, the example in the execution guide, is contended: every student tries to bind it. If another user holds it on your node your server fails to bind and your client silently connects to *theirs*, which appears as `Method not found` rather than a recognisable error. The Section 2 scripts derive their ports from the user id for this reason.

6 Summary

- **Section 1 Q1** multiplies matrices by the Row–Row method with B replicated to every mapper, passing all 9 correctness cases on 4 nodes. Cost is governed by intermediate volume $m \cdot n \cdot p$, not input size; the combiner is what keeps the final merge cheap.
- **Section 2 Q1** computes the full HW2 Q7 analytics in **4.922 s** on 10M records with 4 map tasks ($3.28\times$), and shrinks intermediate data **278 $\times$ below the input** through in-mapper combining. Against MPI it is $1.4\text{--}2.4\times$ slower but uses $39\times$ less memory.
- **Section 2 Q2** sustains **2,999,449 records/s** with 4 workers ($3.75\times$) while answering queries throughout ingestion. Message granularity is the dominant tuning knob at $296\times$; the single-threaded coordinator is the ceiling.
- **Section 3 Q2** implements the full ordering lifecycle with validated transitions, streaming updates and correct gRPC status codes for every required error case, verified across three compute nodes.

All correctness claims in this report are reproducible with the commands above; all performance figures are generated directly from the recorded CSV files.